PROJECT NOTES · FRONTEND

EduSystem Frontend React SPA, step by step

A rebuild guide to the EduSystem React client. It covers the axios client and the Sanctum bearer session, race-free data fetching, role-based routing for admins, students and parents, forms with Laravel validation, and every dashboard screen. Each step shows clean code that was compiled and tested, and flags each bug fixed against the repo.

React 19.3 · React Router 7.18 · Axios 1.20 · Tailwind 3.4 · CRA 5 · Jest 27 · Laravel 13 + Sanctum API · react-hot-toast

Part A · Overview
Architecture, stack, folders, role matrix

Part B · Frontend
API client, session, state, routing, forms, every screen

Part C · Testing
CRA + React Router 7 test setup, unit tests

Part D · Reference
Endpoints, env, version notes, bugs fixed

Source: `education-system/frontend` (branch `main`), with the API contract from `education-system/backend/routes/api.php` · 32 steps · October 2026

Sommaire

Part A · Overview 3

01 What it is & how the pieces fit 3
02 Tech stack & folder structure 3
03 Roles & what each one can do 4

Part B · Frontend 5

04 Setup: CRA, Tailwind & environment 5
05 Talking to the API: base URL, CORS, headers 6
06 Token storage: one module owns it 6
07 HTTP client & interceptors 7
08 Error helpers for Laravel responses 9
09 Service layer: one object per API resource 9
10 Data fetching: `useFetch` without race conditions 11
11 Which state goes where 12
12 AuthContext: the session in one place 13
13 Roles in one config: sidebar and guards share it 14
14 Route map & guards 15
15 Layouts: public, dashboard and auth shells 17
16 UI kit: small, composable components 18
17 Forms & server-side validation 19
18 Course catalog shared by the public site and the dashboard 20
19 Course detail & enrolment 21
20 Course form (admin): create & edit 22
21 Categories & enrollments: CRUD in modals 23
22 Users admin: search, roles, parents 24
23 Role dashboards & charts 24
24 Parent & profile pages 25
25 Logout, session expiry & multiple tabs 26
26 Frontend lifecycle cheat table 26

Part C · Testing 27

27 Test setup with CRA 5 (Jest 27) and React Router 7 27
28 Unit tests: guards, navigation, useFetch, errors 28

Part D · Reference 30

29 API endpoints used by the frontend 30
30 Environment variables & commands 30
31 Version notes & upgrade path 31
32 Bugs & gaps fixed vs. the repo 32

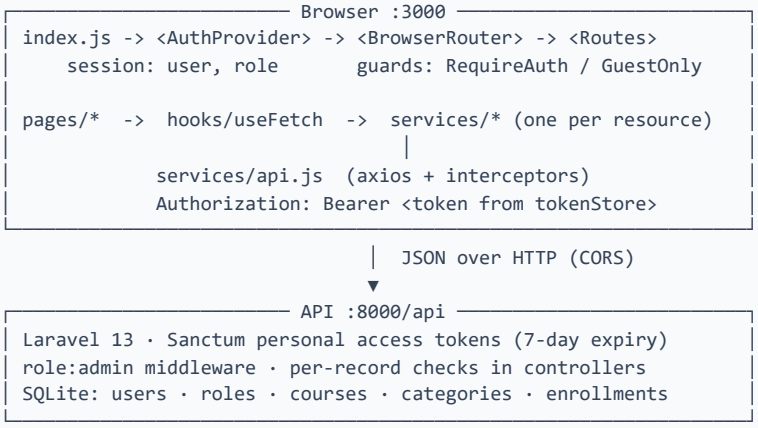
PART A

Overview

What EduSystem is, how the React SPA talks to the Laravel API, the stack and the folder layout.

STEP 01 What it is & how the pieces fit

EduSystem is a React single-page app for three roles: **admin**, **student** and **parent**. It has a public catalog plus one role-aware dashboard. All data comes from the Laravel 13 + Sanctum API in `../backend`.



Concept	How it works here	Where
Session	Bearer token in `localStorage`, user object in React state	`services/session.js`, `context/AuthContext.jsx`
HTTP	One axios instance, auth header + global error toasts	`services/api.js`
Data access	One service per API resource, always returns the body	`services/index.js`
Server state	`useFetch`: loading/error/data, aborts stale requests	`hooks/useFetch.js`
Roles	Role groups shared by the sidebar and the route guards	`config/navigation.jsx`
Routing	Public, guest-only and dashboard trees, nested role guards	`App.jsx`, `components/guards.jsx`
UI	Tailwind + small home-made kit (Button, Modal, Table…)	`components/ui`, `charts.jsx`

STEP 02 Tech stack & folder structure

Versions are the ones installed in `node_modules`, not just the ranges in `package.json`.

Tool	Version	Role here
React / React DOM	19.3	Function components + hooks only
React Router DOM	7.18	Declarative `<Routes>` (library mode, no loaders)
Axios	1.20	Single instance with interceptors
react-hot-toast	2.6	Global notifications, deduplicated by `id`
Tailwind CSS	3.4	PostCSS plugin + `tailwind.config.js` (v3 style)
react-scripts (CRA)	5.0.1	Dev server, build, Jest 27. CRA is deprecated (see Reference)
Testing Library	React 16 · DOM 10	Jest + jsdom through `react-scripts test`

folder structure

```
src/
├── index.js                # root render + <Toaster>
├── App.jsx                 # the whole route tree
├── index.css               # Tailwind layers + .input / .label
├── config/navigation.jsx   # ROLES, dashboardNav, publicNav, navForRole()
├── context/AuthContext.jsx # session state + login/register/logout
├── services/
│   ├── session.js          # token storage + "session expired" event   (new)
│   ├── errors.js           # getErrorMessage, fieldErrors              (new)
│   ├── api.js              # axios instance + interceptors
│   └── index.js            # authService, courseService, ...
├── hooks/useFetch.js       # data loading with abort
├── hooks/useDebounced.js   # shared debounce                           (new)
├── components/
│   ├── guards.jsx          # RequireAuth, GuestOnly, Forbidden
│   ├── layouts/            # PublicLayout, DashboardLayout, AuthShell
│   ├── ui/index.jsx        # Button, Card, Modal, Table, Field, ...
│   ├── catalog.jsx         # course list/filter/card shared by both catalogs
│   ├── charts.jsx          # SVG Donut/Trend/Column/BarList (no chart lib)
│   └── CourseImage.jsx, Logo.jsx, icons.jsx
└── pages/
    ├── public/             # Landing, About, Courses, CourseDetail, Categories
    ├── auth/               # Login, Register
    └── dashboard/          # Overview, Courses, CourseDetail, CourseForm,
                            # Categories, Enrollments, Children, Users, Profile
```

STEP 03 Roles & what each one can do

The frontend hides what a role cannot do. The **backend enforces it** (`role:admin` middleware and checks in the controllers).

Capability	Admin	Student	Parent	Route / endpoint
Browse public catalog	✓	✓	✓	`/courses`, `/categories`
Dashboard overview	platform stats + charts	own progress	children's progress	`/dashboard`
Enroll in a course	—	✓	—	`POST /enrollments`
Drop an enrollment	✓	own	children's (backend rule)	`DELETE /enrollments/:id`
Set progress / grade	✓	—	—	`PUT /enrollments/:id`
Create / edit / delete courses	✓	—	—	`/dashboard/courses/create`
Manage categories	✓	read	read	`/dashboard/categories`
Manage users	✓	—	—	`/dashboard/users`
My children page	—	—	✓	`/dashboard/children`
Edit own profile	✓	✓	✓	`/dashboard/profile`

WHY

Hiding a button is UX, not security. Every rule in this table must also exist on the API. The frontend guard only avoids showing a page that would answer 403 anyway.

PART B

Frontend

From setup to every page: API client, session, data fetching, state, routing and role guards, forms, and each feature screen. Code is the cleaned-up version, compiled and tested.

STEP 04 Setup: CRA, Tailwind & environment

The app is Create React App 5 with Tailwind 3 added through PostCSS. Only variables prefixed with `REACT_APP_` reach the browser bundle.

terminal

```
npm install
cp .env.example .env          # REACT_APP_API_URL=http://127.0.0.1:8000/api
npm start                     # http://localhost:3000 (backend: php artisan serve on :8000)
npm test                      # Jest watch mode
npm run build                 # production bundle in build/
```

tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ['./src/**/*.{js,jsx}', './public/index.html'],
  theme: {
    extend: {
      colors: {
        brand: {
          50: '#eef2ff', 100: '#e0e7ff', 200: '#c7d2fe', 300: '#a5b4fc', 400: '#818cf8',
          500: '#6366f1', 600: '#4f46e5', 700: '#4338ca', 800: '#3730a3', 900: '#312e81',
        },
      },
    },
  },
  plugins: [],
};
```

postcss.config.js

```
module.exports = {
  plugins: { tailwindcss: {}, autoprefixer: {} },
};
```

src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html { scroll-behavior: smooth; }
  body { @apply bg-slate-50 text-slate-800 antialiased; }
  :focus-visible { @apply outline-none ring-2 ring-brand-400 ring-offset-1; }
}

@layer components {
  .input { @apply w-full rounded-lg border border-slate-300 bg-white px-3 py-2 text-sm shadow-sm placeholder:text-
slate-400 focus:border-brand-500 focus:outline-none focus:ring-2 focus:ring-brand-200 disabled:bg-slate-100; }
  .label { @apply mb-1 block text-sm font-medium text-slate-700; }
}
```

VS. THE REPO

The root `README.md` says "Tailwind v4", but the installed version is **3.4.19** (`@tailwind` directives, `tailwind.config.js`, the `tailwindcss` PostCSS plugin). Fix the README, or migrate (Step 31).

WATCH OUT

`.env` values are compiled **into** the JS bundle. Never put secrets in a `REACT_APP_*` variable: anyone can read them in DevTools.

STEP 05 Talking to the API: base URL, CORS, headers

The SPA (:3000) and the API (:8000) are different origins. The browser therefore needs CORS approval from Laravel, and every request must ask for JSON.

Concern	Value	Where it is set
Base URL	`REACT_APP_API_URL` → `http://127.0.0.1:8000/api`	`services/api.js`
CORS origins	`http://localhost:3000`, `http://127.0.0.1:3000`	`backend/config/cors.php` (`CORS_ALLOWED_ORIGINS`)
Auth	`Authorization: Bearer <token>`, no cookies	request interceptor
Accept	`application/json`: Laravel returns 401/422 as JSON instead of redirecting	axios default header
Content-Type	Set by axios for object bodies (`application/json`)	automatic

Mode	How	Pros	Cons
Sanctum **API tokens** (this repo)	Token in JS storage, sent as a header	Simple, works cross-domain and for mobile	Readable by any XSS script; you revoke it with `/auth/logout`
Sanctum **SPA cookies**	HttpOnly session cookie + CSRF cookie	JS cannot read the session; XSS cannot steal it	Same top-level domain; needs `withCredentials` + `/sanctum/csrf-cookie`

WATCH OUT

`127.0.0.1` and `localhost` are **different origins**. Open the app with the same host that `CORS_ALLOWED_ORIGINS` lists, or you will get CORS errors that look like "Cannot reach the server".

STEP 06 Token storage: one module owns it

Only `session.js` reads or writes the token. It also publishes a tiny "session expired" event, so the HTTP layer can end the session without importing React.

src/services/session.js

```
export const TOKEN_KEY = 'token';
const listeners = new Set();

/** The only module that touches the token in localStorage. */
export const tokenStore = {
  get: () => localStorage.getItem(TOKEN_KEY),
  set: (token) => localStorage.setItem(TOKEN_KEY, token),
  clear: () => {
    localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem('user'); // legacy copy written by older builds
  },
};

/** Subscribe to "the API rejected our token". Returns the unsubscribe function. */
export function onSessionExpired(listener) {
```

```
  listeners.add(listener);
  return () => listeners.delete(listener);
}

/** Called by the HTTP client on a 401: clear the token and tell the AuthProvider. */
export function expireSession() {
  tokenStore.clear();
  listeners.forEach((listener) => listener());
}
```

Do	Don't
Keep the token in **one** module (`tokenStore`)	Call `localStorage.getItem('token')` all over the code
Keep the user in React state and reload it from `/auth/me`	Cache the user in `localStorage` (it goes stale and leaks personal data)
Clear the token on 401 and on logout	Keep a token the server already rejected
Escape output (React does it) and avoid `dangerouslySetInnerHTML`	Render HTML from the API: XSS can read `localStorage`
Consider Sanctum SPA cookies for production	Put tokens in URLs or in `REACT_APP_*` env vars

VS. THE REPO

The repo wrote `localStorage.user` on login, on register, in `getProfile` and in `updateProfile` , but never read it. `getCurrentUser()` and `isAuthenticated()` were unused. That was a stale copy of personal data (phone, address, date of birth). It is removed now, and `clear()` also deletes the old key.

STEP 07 HTTP client & interceptors

One axios instance. The request interceptor adds the token. The response interceptor turns failures into a single toast and, on 401, ends the session through the event instead of reloading the page.

src/services/api.js

```
import axios from 'axios';
import toast from 'react-hot-toast';
import { expireSession, tokenStore } from './session';
import { getErrorMessage } from './errors';

const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://127.0.0.1:8000/api',
  // Without Accept: application/json Laravel answers auth/validation errors with redirects
  headers: { Accept: 'application/json' },
});

api.interceptors.request.use((config) => {
  const token = tokenStore.get();
  // Public pages pass { public: true } so they always see what a guest sees
  if (token && !config.public) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

const AUTH_ENDPOINTS = ['/auth/login', '/auth/register'];
const FALLBACK = {
  403: 'You do not have permission to do that.',
  404: 'Resource not found.',
  422: 'Please check the form and try again.',
};

api.interceptors.response.use(
  (response) => response,
  (error) => {
    const { response, config = {} } = error;
```

```
    if (axios.isCancel(error)) {
      // Aborted by useFetch (newer request or unmount): not an error for the user
    } else if (!response) {
      if (!config.silent) toast.error('Cannot reach the server. Check your connection.', { id: 'network' });
    } else if (response.status === 401) {
      // A rejected token ends the session; the AuthProvider clears the user and the guards redirect
      if (tokenStore.get() && !AUTH_ENDPOINTS.includes(config.url)) {
        expireSession();
        toast.error('Your session has expired. Please log in again.', { id: 'session' });
      }
    } else if (!config.silent) {
      toast.error(
        response.status >= 500
          ? 'Server error. Please try again later.'
          : getErrorMessage(error, FALLBACK[response.status])
      );
    }

    return Promise.reject(error);
  }
);

export default api;
```

Case	Toast / action	Notes
Aborted (`CanceledError`)	nothing	useFetch cancelled a stale request
No response	“Cannot reach the server” (`id: network`, deduped)	unless `silent`
401 with a token	`expireSession()` + “session expired”	guards redirect to `/login` with `from`
401 on login/register	nothing here	the page shows its own result
403 / 404 / 422	first validation error, or `message`, or a fallback	unless `silent`
5xx	“Server error. Please try again later.”	unless `silent`

Request option	Effect
`{ public: true }`	Do not send the token: public pages show what a guest sees (no unpublished courses for an admin)
`{ silent: true }`	No global toast: the caller handles the error (logout, public course page, `/auth/me`)
`{ signal }`	AbortController signal passed by `useFetch`

VS. THE REPO

The repo did `window.location.assign('/login')` on 401. That is a full reload: React state is lost, the “return to” location (`from`) is lost, and AuthContext still held the old user. Aborted requests also had no response, so they would have shown the network toast. Both are fixed.

STEP 08 Error helpers for Laravel responses

Laravel validation errors are `{ message, errors: { field: [msg, …] } }`. Two pure functions cover every page: one message for toasts, one map for form fields.

src/services/errors.js

```
/** Human-readable message from an axios error (first validation error wins). */
export function getErrorMessage(error, fallback = 'Something went wrong.') {
  const data = error?.response?.data;
  const firstFieldError = data?.errors && Object.values(data.errors)[0]?.[0];
  if (firstFieldError) return firstFieldError;
  if (data?.message) return data.message;
  return error?.request && !error.response ? 'Cannot reach the server.' : fallback;
}

/** Laravel 422 `{ errors: { field: [msg, ...] } }` -> `{ field: msg }` for <Field error={...}>. */
export function fieldErrors(error) {
  const errors = error?.response?.data?.errors || {};
  return Object.fromEntries(Object.entries(errors).map(([field, messages]) => [field, messages[0]]));
}
```

response shape

```
// HTTP 422 from POST /auth/register
{
  "message": "The email has already been taken. (and 1 more error)",
  "errors": {
    "email":    ["The email has already been taken."],
    "password": ["The password field must be at least 8 characters."]
  }
}
// fieldErrors(err)  ->  { email: "The email has already been taken.", password: "The password field must..." }
```

VS. THE REPO

The repo copied `Object.fromEntries(Object.entries(fe).map(([k, v]) => [k, v[0]]))` into five pages. It is now `setErrors(fieldErrors(err))`.

STEP 09 Service layer: one object per API resource

Pages never call axios directly. Each service returns the response **body**. A small `resource()` factory covers the three plain CRUD resources.

src/services/index.js

```
import api from './api';
import { tokenStore } from './session';

/** Every service returns the response body, never the axios response. */
const data = (request) => request.then((res) => res.data);

/** Standard Laravel apiResource: index / show / store / update / destroy. */
const resource = (path) => ({
  getAll: ({ params, ...config } = {}) => data(api.get(path, { params, ...config })),
  getById: (id, config) => data(api.get(`${path}/${id}`, config)),
  create: (body) => data(api.post(path, body)),
  update: (id, body) => data(api.put(`${path}/${id}`, body)),
  delete: (id) => data(api.delete(`${path}/${id}`)),
});

export const courseService = resource('/courses');
export const categoryService = resource('/categories');
export const userService = resource('/users');
```

```
export const authService = {
  async login(email, password) {
    const res = await data(api.post('/auth/login', { email, password }));
    tokenStore.set(res.token);
    return res;
  },

  async register(payload) {
    const res = await data(api.post('/auth/register', payload));
    tokenStore.set(res.token);
    return res;
  },

  async logout() {
    try {
      // Revoke the token server-side while it is still attached to the request
      await api.post('/auth/logout', null, { silent: true });
    } catch {
      // Ignore: local state is cleared regardless
    } finally {
      tokenStore.clear();
    }
  },

  me: (config) => data(api.get('/auth/me', config)),
  updateProfile: (body) => data(api.put('/auth/profile', body)),
  getRoles: (config) => data(api.get('/auth/roles', config)),
  getParents: (config) => data(api.get('/auth/parents', config)),
};

export const enrollmentService = {
  getAll: (config) => data(api.get('/enrollments', config)),
  enroll: (courseId, studentId) =>
    data(api.post('/enrollments', { course_id: courseId, ...(studentId && { student_id: studentId }) })),
  update: (id, body) => data(api.put(`/enrollments/${id}`, body)),
  drop: (id) => data(api.delete(`/enrollments/${id}`)),
};

export const dashboardService = {
  getStats: (config) => data(api.get('/dashboard', config)),
};
```

Service	Base path	Methods
`courseService`	`/courses`	getAll({ params: { search, category_id, level } }), getById, create, update, delete
`categoryService`	`/categories`	same five methods
`userService`	`/users`	same five methods (admin; `params: { search, role }`)
`authService`	`/auth/*`	login, register, logout, me, updateProfile, getRoles, getParents
`enrollmentService`	`/enrollments`	getAll, enroll(courseId, studentId?), update, drop
`dashboardService`	`/dashboard`	getStats (shape depends on the role)

> **TIP**
> Every GET takes an axios config object, so a page can pass `signal`, `public` or `silent` without the service having to know about them.

STEP 10 Data fetching: `useFetch` without race conditions

Every page loads data through one hook. It passes an `AbortSignal` to the loader and ignores any result that arrives after a newer request started.

src/hooks/useFetch.js

```
import { useCallback, useEffect, useRef, useState } from 'react';

/**
 * Run `loader({ signal })` on mount and whenever `deps` change.
 * A newer run aborts the previous one, so a slow old response can never overwrite fresh data.
 */
export default function useFetch(loader, deps = []) {
  const [state, setState] = useState({ data: null, loading: true, error: null });
  const loaderRef = useRef(loader);
  const controllerRef = useRef(null);

  // Always call the latest closure without re-running the effect on every render
  useEffect(() => {
    loaderRef.current = loader;
  });

  const load = useCallback(() => {
    controllerRef.current?.abort();
    const controller = new AbortController();
    controllerRef.current = controller;
    setState((s) => ({ ...s, loading: true }));

    return loaderRef.current({ signal: controller.signal }).then(
      (data) => !controller.signal.aborted && setState({ data, loading: false, error: null }),
      (error) => !controller.signal.aborted && setState((s) => ({ ...s, loading: false, error }))
    );
  }, []);

  useEffect(() => {
    load();
    return () => controllerRef.current?.abort();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, deps);

  const setData = useCallback(
    (next) => setState((s) => ({ ...s, data: typeof next === 'function' ? next(s.data) : next })),
    []
  );

  return { ...state, reload: load, setData };
}
```

```
typing "re" ... "react" (debounced)
  t0  GET /courses?search=re      ──────────────(slow 900 ms)──────────► 3 results
  t1  GET /courses?search=react   ──(fast 200 ms)──► 1 result ✓ shown
                                                     old repo: the 3 results overwrite it ✗
                                                     now: request t0 is aborted → ignored ✓
```

src/hooks/useDebounced.js

```
import { useEffect, useState } from 'react';

/** Return `value` once it has stopped changing for `ms` (search boxes, filters). */
export default function useDebounced(value, ms = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), ms);
    return () => clearTimeout(t);
```

```
  }, [value, ms]);
  return debounced;
}
```

usage

```
// usage: the loader receives { signal } and forwards it to the service
const { data: course, loading, error, reload } = useFetch(
  ({ signal }) => courseService.getById(id, { signal }),
  [id]                                  // re-run when the route param changes
);
```

VS. THE REPO

In the repo, `useFetch` had no cancellation. With the debounced catalog search and the Users search, a slow earlier response could replace newer results. It could also call `setState` after navigation.

NOTE

StrictMode runs effects twice in development: the first request is aborted right away. Two requests in the Network tab, one of them “canceled”, is normal.

STEP 11 Which state goes where

No Redux. Each kind of state has one home, and the server stays the source of truth for data.

State	Home	Why
Session: user, role, loading	`AuthContext`	needed by guards, layouts and many pages
Server data: courses, enrollments, stats	`useFetch` per page (+ `reload()` after a mutation)	fresh on every visit; no global cache to invalidate
Route data: `:id`, `?category_id=`	React Router `useParams` / `useSearchParams`	shareable links, works with Back
Form inputs and field errors	`useState` in the page	local and short-lived
Modal open/closed + what it edits	`useState(null | object)`	`null` = closed, object = open (with `id` when editing)
Notifications	`react-hot-toast`	fire-and-forget from anywhere, interceptor included
Token	`tokenStore` (localStorage)	survives reloads, shared across tabs

TIP

If pages start sharing server data (e.g. the same course list in three places) or you want caching and background refetching, swap `useFetch` for **TanStack Query**: `useQuery({ queryKey: ['courses', filters], queryFn: ({ signal }) => courseService.getAll({ params: filters, signal }) })`. The services stay the same.

STEP 12 AuthContext: the session in one place

The provider restores the session from the token, exposes role flags, and follows two outside events: the API rejecting the token, and another tab logging in or out.

src/context/AuthContext.jsx

```
import React, { createContext, useCallback, useContext, useEffect, useMemo, useState } from 'react';
import { authService } from '../services';
import { onSessionExpired, tokenStore, TOKEN_KEY } from '../services/session';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(() => !!tokenStore.get());

  /** Rebuild the session from the stored token (page load, or another tab changed it). */
  const restore = useCallback(async () => {
    if (!tokenStore.get()) {
      setUser(null);
      setLoading(false);
      return;
    }
    try {
      setUser(await authService.me({ silent: true }));
    } catch {
      // 401 is handled by the interceptor (expireSession); a network blip keeps the token
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    restore();
  }, [restore]);

  // Token rejected by the API -> drop the user; RequireAuth then redirects to /login
  useEffect(() => onSessionExpired(() => setUser(null)), []);

  // Login or logout in another tab -> follow it here
  useEffect(() => {
    const onStorage = (e) => e.key === TOKEN_KEY && restore();
    window.addEventListener('storage', onStorage);
    return () => window.removeEventListener('storage', onStorage);
  }, [restore]);

  const value = useMemo(() => {
    const role = user?.role?.name;
    return {
      user,
      role,
      loading,
      isAuthenticated: !!user,
      isAdmin: role === 'admin',
      isStudent: role === 'student',
      isParent: role === 'parent',
      hasRole: (...roles) => roles.includes(role),
      login: async (email, password) => {
        const res = await authService.login(email, password);
        setUser(res.user);
        return res;
      },
      register: async (payload) => {
        const res = await authService.register(payload);
        setUser(res.user);
```

```
        return res;
      },
      logout: async () => {
        await authService.logout();
        setUser(null);
      },
      updateUser: setUser,
    };
  }, [user, loading]);

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within an AuthProvider');
  return context;
}
```

Event	What happens	Result
Page load with token	`loading=true` → `GET /auth/me` (silent) → `user` set	guards show a loader, never flash the login page
Page load, no token	`loading=false`, `user=null`	no request at all
`login()` / `register()`	service stores the token → `setUser(res.user)`	`GuestOnly` redirects
Any 401 later	interceptor → `expireSession()` → `setUser(null)`	`RequireAuth` → `/login`
Token changes in another tab	`storage` event → `restore()`	both tabs agree
`logout()`	revoke on server (silent) → clear token → `setUser(null)`	even if the API is down

VS. THE REPO

The repo exposed `isAdmin()`, `isStudent()`, `isParent()` as functions and rebuilt the context value on every render. They are now booleans (`isAdmin`), plus `role`, inside a `useMemo`, so consumers only re-render when the user or the loading flag changes.

STEP 13 Roles in one config: sidebar and guards share it

The role groups live in one file. The sidebar and the route guards both read them, so a page cannot be visible in the menu but blocked by its route, or the reverse.

src/config/navigation.jsx

```
/** Role groups shared by the sidebar and the route guards in App.jsx so the two cannot drift apart. */
export const ROLES = {
  admin: ['admin'],
  parent: ['parent'],
  enrollments: ['admin', 'student', 'parent'],
};

/** Single source of truth for role-aware navigation. `roles` omitted = every authenticated user. */
export const dashboardNav = [
  {
    heading: 'Main',
    items: [
      { to: '/dashboard', label: 'Overview', icon: 'squares', end: true },
      { to: '/dashboard/courses', label: 'Courses', icon: 'book' },
      { to: '/dashboard/categories', label: 'Categories', icon: 'tag' },
      { to: '/dashboard/enrollments', label: 'Enrollments', icon: 'list', roles: ROLES.enrollments },
      { to: '/dashboard/children', label: 'My children', icon: 'users', roles: ROLES.parent },
```

```
    ],
  },
  {
    heading: 'Administration',
    roles: ROLES.admin,
    items: [
      { to: '/dashboard/users', label: 'Users', icon: 'users', roles: ROLES.admin },
    ],
  },
  {
    heading: 'Account',
    items: [
      { to: '/dashboard/profile', label: 'Profile', icon: 'user' },
    ],
  },
];

export const publicNav = [
  { to: '/', label: 'Home', end: true },
  { to: '/courses', label: 'Courses' },
  { to: '/categories', label: 'Categories' },
  { to: '/about', label: 'About' },
];

export const canSee = (entry, role) => !entry.roles || entry.roles.includes(role);

export const navForRole = (role) =>
  dashboardNav
    .filter((g) => canSee(g, role))
    .map((g) => ({ ...g, items: g.items.filter((i) => canSee(i, role)) }))
    .filter((g) => g.items.length);
```

> **TIP**
> To add a page for one role, add its group to `ROLES` (if new), one item to `dashboardNav` and one nested `<RequireAuth roles={ROLES.x}>` route. That is three lines in two files.

STEP 14 Route map & guards

`App.jsx` has four trees: marketing pages, the public catalog, guest-only auth pages, and the dashboard. Guards are layout routes that render `<Outlet />` or redirect.

src/App.jsx (excerpt)

```
<Routes>
  {/* Marketing pages: full-width sections (the header adapts to the session) */}
  <Route element={<PublicLayout contained={false} />}>
    <Route path="/" element={<Landing />} />
    <Route path="/about" element={<About />} />
  </Route>

  {/* Public catalog: read-only for everyone, never shows management actions */}
  <Route element={<PublicLayout />}>
    <Route path="/courses" element={<PublicCourses />} />
    <Route path="/courses/:id" element={<PublicCourseDetail />} />
    <Route path="/categories" element={<PublicCategories />} />
  </Route>

  {/* Guest-only: signed-in users are bounced to /dashboard */}
  <Route element={<GuestOnly />}>
    <Route path="/login" element={<Login />} />
    <Route path="/register" element={<Register />} />
  </Route>

  {/* Authenticated area: any signed-in user; nested guards narrow it by role (403 otherwise) */}
```

```
  <Route path="/dashboard" element={<RequireAuth />}>
    <Route element={<DashboardLayout />}>
      <Route index element={<Overview />} />
      <Route path="courses" element={<Courses />} />
      <Route path="courses/:id" element={<CourseDetail />} />
      <Route path="categories" element={<Categories />} />
      <Route path="profile" element={<Profile />} />

      <Route element={<RequireAuth roles={ROLES.enrollments} />}>
        <Route path="enrollments" element={<Enrollments />} />
      </Route>
      <Route element={<RequireAuth roles={ROLES.parent} />}>
        <Route path="children" element={<Children />} />
      </Route>
      <Route element={<RequireAuth roles={ROLES.admin} />}>
        <Route path="users" element={<Users />} />
        <Route path="courses/create" element={<CourseForm />} />
        <Route path="courses/:id/edit" element={<CourseForm />} />
      </Route>
      <Route path="*" element={<NotFound home="/dashboard" />} />
    </Route>
  </Route>

  <Route element={<PublicLayout />}>
    <Route path="*" element={<NotFound />} />
  </Route>
</Routes>
```

src/components/guards.jsx

```
import React from 'react';
import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { LinkButton, PageLoader } from './ui';

/** Requires a signed-in user; optionally restricts to `roles`. Renders nested routes via <Outlet />. */
export function RequireAuth({ roles }) {
  const { isAuthenticated, loading, role } = useAuth();
  const location = useLocation();

  if (loading) return <PageLoader />;
  if (!isAuthenticated) return <Navigate to="/login" state={{ from: location }} replace />;
  if (roles && !roles.includes(role)) return <Forbidden />;
  return <Outlet />;
}

/** Guest-only pages (login, register). Once signed in, go back to where the user came from. */
export function GuestOnly() {
  const { isAuthenticated, loading } = useAuth();
  const from = useLocation().state?.from;

  if (loading) return <PageLoader />;
  if (isAuthenticated) return <Navigate to={from ? `${from.pathname}${from.search}` : '/dashboard'} replace />;
  return <Outlet />;
}

export function Forbidden() {
  return (
    <div className="py-24 text-center">
      <p className="text-6xl font-bold text-rose-500">403</p>
      <p className="mt-2 text-slate-600">You do not have permission to view this page.</p>
      <LinkButton to="/dashboard" className="mt-6">Back to dashboard</LinkButton>
    </div>
  );
}
```

Path	Wrapper	Who
`/` , `/about`	PublicLayout (full width)	everyone
`/courses` , `/courses/:id` , `/categories`	PublicLayout	everyone, always as a guest (`public: true`)
`/login` , `/register`	GuestOnly → AuthShell	guests; signed-in users go to `from` or `/dashboard`
`/dashboard` , `courses` , `courses/:id` , `categories` , `profile`	RequireAuth → DashboardLayout	any signed-in user
`/dashboard/enrollments`	+ `RequireAuth roles={ROLES.enrollments}`	admin, student, parent
`/dashboard/children`	+ `roles={ROLES.parent}`	parent
`/dashboard/users` , `courses/create` , `courses/:id/edit`	+ `roles={ROLES.admin}`	admin (403 page otherwise)

NOTE

`courses/create` and `courses/:id` do not clash: React Router ranks a static segment above a dynamic one, whatever the declaration order.

VS. THE REPO

`Login` called `navigate(from)` while `GuestOnly` also redirected to `/dashboard` as soon as the user was set. Two navigations raced. Now only `GuestOnly` redirects (to `from`), and Login and Register just call `login()` / `register()` .

STEP 15 Layouts: public, dashboard and auth shells

There are three shells. Each is a layout route, so pages only render their own content.

Layout	What it provides	File
`PublicLayout`	sticky header (nav + session-aware actions), footer; `contained` prop for full-width marketing	`components/layouts/PublicLayout.jsx`
`DashboardLayout`	fixed sidebar ≥ lg, drawer below lg, top bar with role + avatar	`components/layouts/DashboardLayout.jsx`
`AuthShell`	split screen: brand panel + form column (`wide` for Register)	`components/layouts/AuthShell.jsx`

src/components/layouts/DashboardLayout.jsx (excerpt)

```
export default function DashboardLayout() {
  const [open, setOpen] = useState(false);
  const { user } = useAuth();
  const { pathname } = useLocation();

  // Close the mobile drawer on navigation and lock body scroll while it is open
  useEffect(() => {
    setOpen(false);
  }, [pathname]);
  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : "";
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);

  return (
    <div className="min-h-screen bg-slate-50">
```

```
      <aside className="fixed inset-y-0 left-0 z-30 hidden w-64 bg-slate-900 lg:block">
        <SidebarContent />
      </aside>
```

> **WHY**
> `AuthActions` returns `null` while `loading`. Without that, a signed-in user would see "Sign in / Get started" flash for a moment on every reload.

STEP 16 UI kit: small, composable components

A home-made kit in `components/ui/index.jsx` keeps the Tailwind class lists in one place. Pages compose these components and never repeat the long class strings.

Component	API
`Button`	variant (primary, secondary, danger, ghost), `loading` → spinner + disabled
`LinkButton`	same look on a router `Link`, or `as="button"`
`Card`, `PageHeader`, `SectionTitle`	layout blocks
`Field`	label + child input + `hint` or `error`
`Table`	`columns=[{ key, header, render? }]`, empty row
`Modal`, `ConfirmModal`	Escape and backdrop close, footer slot
`Badge`, `ProgressBar`, `StatCard`, `Avatar`, `Skeleton`, `EmptyState`, `PageLoader`	display

src/components/ui/index.jsx (excerpt)

```
export function Modal({ open, title, onClose, children, footer }) {
  useEffect(() => {
    if (!open) return undefined;
    const onKey = (e) => e.key === 'Escape' && onClose();
    document.addEventListener('keydown', onKey);
    return () => document.removeEventListener('keydown', onKey);
  }, [open, onClose]);

  if (!open) return null;
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4" role="dialog" aria-modal="true">
      <div className="absolute inset-0 bg-slate-900/50" onClick={onClose} />
      <div className="relative max-h-[90vh] w-full max-w-lg overflow-y-auto rounded-xl bg-white p-6 shadow-xl">
        <h2 className="mb-4 text-lg font-semibold text-slate-900">{title}</h2>
        {children}
        {footer && <div className="mt-6 flex justify-end gap-2">{footer}</div>}
      </div>
    </div>
  );
}

export function ConfirmModal({ open, title = 'Are you sure?', message, confirmText = 'Confirm', loading, onConfirm,
onClose }) {
  return (
    <Modal
      open={open}
      title={title}
      onClose={onClose}
      footer={
        <>
          <Button variant="secondary" onClick={onClose}>Cancel</Button>
          <Button variant="danger" loading={loading} onClick={onConfirm}>{confirmText}</Button>
```

```
        </>
      }
    >
      <p className="text-sm text-slate-600">{message}</p>
    </Modal>
  );
}
```

VS. THE REPO

`Avatar` defaulted `name = '?'`, but a `null` name skips the default and `null.split` crashed. It now uses `(name || '?')`.

WATCH OUT

The modal has no focus trap and does not return focus on close. For full accessibility, use the native `<dialog>` element with `showModal()`, which handles both.

STEP 17 Forms & server-side validation

Every form uses the same pattern: controlled inputs in `useState`, submit, then show 422 field errors under the matching `<Field>`. The API is the validator; HTML attributes (`required`, `min`) only help the user earlier.

src/pages/auth/Login.jsx (excerpt)

```
function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    try {
      const data = await login(email, password);
      // No navigate() here: <GuestOnly> sees the new session and redirects to `from`
      toast.success(`Welcome back, ${data.user.name}!`);
    } catch {
      // The 422 "credentials are incorrect" toast comes from the API interceptor
    } finally {
      setLoading(false);
    }
  };
```

src/pages/auth/Register.jsx (excerpt)

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setErrors({});
  setLoading(true);
  try {
    const payload = { ...form };
    Object.keys(payload).forEach((k) => payload[k] === '' && delete payload[k]);
    if (payload.role !== 'student') { delete payload.parent_id; delete payload.date_of_birth; }
    await register(payload);
    toast.success('Account created. Welcome!'); // <GuestOnly> redirects to /dashboard
  } catch (err) {
    setErrors(fieldErrors(err));
  } finally {
    setLoading(false);
  }
};
```

Rule	How
Send only what the API expects	drop empty strings; drop `parent_id` / `date_of_birth` unless student
Show errors per field	`<Field error={errors.email}>`, cleared on each submit
Disable while sending	`<Button loading={saving}>` blocks double submits
Never trust the select	`role` is a select, but the API must refuse `admin` on register

VS. THE REPO
Register showed no error under “Confirm password”, so the `password_confirmation` field error was invisible. It is wired now.

STEP 18 Course catalog shared by the public site and the dashboard

One hook and three components serve both catalogs. Only the link target and the `public` flag differ. No role logic lives in the shared code.

src/components/catalog.jsx (excerpt)

```
const emptyFilters = { search: '', category_id: '', level: '' };

/** Filters (debounced search), category list and course list. `publicView` ignores the session entirely. */
export function useCourseCatalog({ publicView = false } = {}) {
  const [searchParams] = useSearchParams();
  const cfg = publicView ? { public: true } : {};
  const [filters, setFilters] = useState({ ...emptyFilters, category_id: searchParams.get('category_id') || '' });
  const debounced = useDebounced(filters); // one request per pause, not per keystroke

  const { data: categories } = useFetch(({ signal }) => categoryService.getAll({ ...cfg, signal }), []);
  const { data: courses, loading } = useFetch(({ signal }) => {
    const params = Object.fromEntries(Object.entries(debounced).filter(([, v]) => v !== ''));
    return courseService.getAll({ params, ...cfg, signal });
  }, [debounced]);

  return { filters, setFilters, clear: () => setFilters(emptyFilters), categories: categories || [], courses,
loading };
}
```

	Public `/courses`	Dashboard `/dashboard/courses`
Token sent	no (`public: true`)	yes
Unpublished courses	never	admins see them, with an “Unpublished” badge
Card links to	`/courses/:id`	`/dashboard/courses/:id`
Extra actions	none	“New course” for admins

TIP
The category filter is read from `?category_id=` once, on mount. To make every filter shareable and Back-friendly, store all filters in `useSearchParams` instead of `useState` .

STEP 19 Course detail & enrolment

The dashboard course page decides which action to show from the role, the current enrolment (`course.enrollment`, sent by the API for the signed-in user) and the capacity.

src/pages/dashboard/CourseDetail.jsx (excerpt)

```
function CourseDetail() {
  const { id } = useParams();
  const { isAdmin, isStudent } = useAuth();
  const navigate = useNavigate();
  const { data: course, loading, reload } = useFetch(({ signal }) => courseService.getById(id, { signal }), [id]);
  const [enrolling, setEnrolling] = useState(false);
  const [confirmDelete, setConfirmDelete] = useState(false);
  const [deleting, setDeleting] = useState(false);

  if (loading && !course) return <PageLoader text="Loading course..." />;
  if (!course) return <EmptyState title="Course not found" action={<Link to="/dashboard/courses" className="text-
brand-700 hover:underline">Back to courses</Link>} />;

  const enrollment = course.enrollment;
  const canEnroll = isStudent && !enrollment && course.is_published;
  const full = course.max_students && course.enrollments_count >= course.max_students;

  const enroll = async () => {
    setEnrolling(true);
    try {
      await enrollmentService.enroll(course.id);
      toast.success('You are now enrolled in this course!');
      reload();
    } catch (e) { /* toast shown by interceptor */ } finally { setEnrolling(false); }
  };
```

Who / state	Sidebar shows
Student, not enrolled, published, not full	“Enroll now”
Student, course full	disabled “Course full”
Student, already enrolled	progress block + “View my enrollments”
Student, unpublished	“This course is not open for enrollment.”
Parent	“Only student accounts can enroll in courses.”
Admin	Edit + Delete (with confirm modal)

WATCH OUT

The “full” check is only a hint, because two students can click at the same time. The API must check `max_students` inside the enrolment transaction.

STEP 20 Course form (admin): create & edit

One component handles `/courses/create` and `/courses/:id/edit`. Two pure mappers keep the API ↔ form conversion out of the JSX.

src/pages/dashboard/CourseForm.jsx (excerpt)

```
/** API course -> form state ('' for empty inputs, never null/undefined). */
const toForm = (c) => ({
  title: c.title, description: c.description || '', category_id: c.category_id, level: c.level,
  price: c.price, duration_hours: c.duration_hours || '', max_students: c.max_students || '',
  image: c.image || '', is_published: !!c.is_published,
});

/** Form state -> API payload (numbers as numbers, empty optional fields as null). */
const toPayload = (form) => ({
  ...form,
  price: Number(form.price),
  duration_hours: form.duration_hours === '' ? null : Number(form.duration_hours),
  max_students: form.max_students === '' ? null : Number(form.max_students),
  image: form.image || null,
});

function CourseForm() {
  const { id } = useParams();
  const editing = Boolean(id);
  const { user } = useAuth();
  const navigate = useNavigate();
  const [form, setForm] = useState(empty);
  const [errors, setErrors] = useState({});
  const [saving, setSaving] = useState(false);

  const { data: categories } = useFetch(({ signal }) => categoryService.getAll({ signal }), []);
  const { data: course, loading, error } = useFetch(
    ({ signal }) => (editing ? courseService.getById(id, { signal }) : Promise.resolve(null)),
    [id]
  );

  useEffect(() => {
    if (course) setForm(toForm(course));
  }, [course]);
  useEffect(() => {
    if (error) navigate('/dashboard/courses', { replace: true });
  }, [error, navigate]);

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm((f) => ({ ...f, [name]: type === 'checkbox' ? checked : value }));
  };

  const submit = async (e) => {
    e.preventDefault();
    setErrors({});
    setSaving(true);
    try {
      const payload = toPayload(form);
      // Only a new course gets the current admin as instructor; an edit keeps the original one
      const saved = editing
        ? await courseService.update(id, payload)
        : await courseService.create({ ...payload, instructor_id: user.id });
      toast.success(editing ? 'Course updated.' : 'Course created.');
      navigate(`/dashboard/courses/${saved.id}`);
    } catch (err) {
      setErrors(fieldErrors(err));
    } finally {
      setSaving(false);
```

```
    }
  };

  if (editing && loading) return <PageLoader />;
```

VS. THE REPO

The repo always sent `instructor_id: user.id`. Editing a course therefore silently made the editing admin its instructor. The backend accepts `instructor_id` as `sometimes` on update, so it is now only sent on create. `price` is also sent as a number now, not as the input's string.

TIP

`type="url"` on the image field gives a browser-side check. The API should still validate it as `url`, and ideally only allow `https`.

STEP 21 Categories & enrollments: CRUD in modals

List pages edit in place with a modal. The state is `null` when the modal is closed, and the object being edited when it is open. On success: toast, close, `reload()`.

src/pages/dashboard/Categories.jsx (excerpt)

```
const save = async () => {
  setBusy(true);
  setErrors({});
  try {
    const { id, ...payload } = form;
    if (id) await categoryService.update(id, payload); else await categoryService.create(payload);
    toast.success(id ? 'Category updated.' : 'Category created.');
    setForm(null);
    reload();
  } catch (err) {
    setErrors(fieldErrors(err));
  } finally { setBusy(false); }
};
```

src/pages/dashboard/Enrollments.jsx (excerpt)

```
const saveEdit = async () => {
  setBusy(true);
  try {
    await enrollmentService.update(editing.id, {
      progress: Number(edit.progress),
      grade: edit.grade === '' ? null : Number(edit.grade),
    });
    toast.success('Enrollment updated.');
    setEditing(null);
    reload();
  } catch (e) { /* interceptor toast */ } finally { setBusy(false); }
};
```

Role	`/dashboard/enrollments` shows	Actions
Admin	all enrollments + Student column	Edit progress/grade, Drop
Student	own enrollments	Drop (active only)
Parent	children's enrollments	Drop (if the API allows it)

NOTE

Dropped enrollments are filtered out on the client: `status !== 'dropped'`. Progress 100 marks the course completed on the server, so the page reloads instead of guessing the new status.

STEP 22 Users admin: search, roles, parents

This is a debounced server-side search, with one modal for create and edit. The dropdowns must not depend on what the table is currently showing.

src/pages/dashboard/Users.jsx (excerpt)

```
function Users() {
  const { user: me } = useAuth();
  const [filters, setFilters] = useState({ search: '', role: '' });
  const query = useDebounced(filters);
  const [form, setForm] = useState(null); // null = closed; object = open (with id when editing)
  const [errors, setErrors] = useState({});
  const [deleting, setDeleting] = useState(null);
  const [busy, setBusy] = useState(false);

  const { data: users, loading, reload } = useFetch(({ signal }) => {
    const params = Object.fromEntries(Object.entries(query).filter(([, v]) => v));
    return userService.getAll({ params, signal });
  }, [query]);

  // Independent of the table filters, so the dropdowns are complete whatever is being searched
  const { data: parents } = useFetch(({ signal }) => authService.getParents({ signal }), []);
  const { data: publicRoles } = useFetch(({ signal }) => authService.getRoles({ signal }), []);
  // /auth/roles hides "admin"; the signed-in admin carries that role object, so add it from there
  const roles = [...(publicRoles || []), me.role].filter((r, i, all) => r && all.findIndex((x) => x.id === r.id)
=== i);

  const openCreate = () => { setErrors({}); setForm(blank); };
```

VS. THE REPO

The repo built the **parent dropdown** from the filtered table rows. Filtering by “student” or searching a name emptied it, so editing a student showed “None” and could unlink their parent. The **admin role** option was also taken from the rows and disappeared under the same filters. Now parents come from `/auth/parents`, and the admin role comes from the signed-in admin's own `role`.

WATCH OUT

The API should refuse to delete the last admin, and refuse to let an admin change their own role. The UI disables the role select for yourself, but that is UX only.

STEP 23 Role dashboards & charts

`/dashboard` calls one endpoint, whose shape depends on the role, then renders one of three dashboards. The charts are small SVG components, so there is no chart dependency.

src/pages/dashboard/Overview.jsx (excerpt)

```
function Dashboard() {
  const { user } = useAuth();
  const role = user?.role?.name;

  // Admin additionally pulls all enrollments (trend chart) and categories (distribution chart)
  const { data, loading } = useFetch(async ({ signal }) => {
    const stats = await dashboardService.getStats({ signal });
    if (role !== 'admin') return { stats };
    const [enrollments, categories] = await Promise.all([enrollmentService.getAll({ signal }),
categoryService.getAll({ signal })]);
    return { stats, enrollments, categories };
  }, [role]);
```

Role	Stat cards	Charts / lists
Admin	students, courses, active enrollments, completion rate	6-month trend, status donut, popular courses, courses per category, recent enrollments
Student	enrolled, completed, in progress, average grade	progress per course, status donut, course cards
Parent	children, enrollments, completed, in progress	average progress per child, status donut, child cards

WATCH OUT

The admin trend chart downloads **every enrollment** just to count them per month. That is fine with demo data, but slow at scale. Add `enrollments_per_month` to `GET /dashboard` and drop the extra request.

WHY

`charts.jsx` uses fixed colours per status (`chartColors.active/completed/dropped`), so "completed" is the same green in every chart.

STEP 24 Parent & profile pages

`/dashboard/children` reuses the parent's dashboard payload. `/dashboard/profile` updates the user and merges the response back into the session.

src/pages/dashboard/Children.jsx (excerpt)

```
// per child: enrollments of that child, without dropped ones
const enrollments = data.enrollments.filter((e) => e.student_id === c.student.id && e.status !== 'dropped');

<div className="text-sm text-slate-600">
  {c.total_enrolled} enrolled · {c.completed} completed ·
  avg grade {c.average_grade != null ? `${Math.round(c.average_grade)}%` : '—'}
</div>
```

src/pages/dashboard/Profile.jsx (excerpt)

```
const submit = async (e) => {
  e.preventDefault();
  setErrors({});
  setSaving(true);
  const payload = { ...form, date_of_birth: form.date_of_birth || null };
  if (!payload.password) { delete payload.password; delete payload.password_confirmation; }
  try {
    const updated = await authService.updateProfile(payload);
    updateUser({ ...user, ...updated });
    setForm({ ...form, password: '', password_confirmation: '' });
    toast.success('Profile updated.');
  } catch (err) {
    setErrors(fieldErrors(err));
  } finally {
    setSaving(false);
  }
};
```

VS. THE REPO

`Math.round(null)` is `0` , so a child with no grade yet showed "avg grade 0%". That looks like failing. It now shows "—", like the Overview page does.

WHY

Password fields are removed from the payload when they are empty, so saving the profile never resets the password to an empty string.

STEP 25 Logout, session expiry & multiple tabs

Three ways a session ends. All of them meet at the same point: no token, `user = null`, and the guards do the redirect.

Trigger	Flow	Result
User clicks Log out	`POST /auth/logout` (silent) → `tokenStore.clear()` → `setUser(null)` → navigate `/login`	token revoked on the server
Token expired or revoked (7 days)	any request → 401 → `expireSession()` → listener `setUser(null)` → RequireAuth → `/login` with `from`	after login, back to the same page
Logout in tab A	tab B gets a `storage` event for `token` → `restore()` → no token → `user=null`	tab B leaves the dashboard at once
Login in tab A	tab B: `storage` → `restore()` → `/auth/me` → user set	tab B's login page redirects

VS. THE REPO

The repo had no cross-tab sync. After logging out in one tab, the other tab kept showing the dashboard until its next request failed.

STEP 26 Frontend lifecycle cheat table

The whole app on one page: what runs, in which order, and where.

Moment	What happens	Where
Boot	`index.js` renders `<App/>` + `<Toaster/>`	index.js
Session restore	token? → `GET /auth/me` (silent) → user/role; else guest	AuthContext
Navigate	`<Routes>` picks a tree → guard → layout → page	App.jsx, guards.jsx
Guard	loading → loader · no user → `/login` + from · wrong role → 403	guards.jsx
Page data	`useFetch(({signal}) => service.x({signal}), deps)`	hooks/useFetch.js
Request	interceptor adds Bearer (unless `public`)	services/api.js
Error	interceptor toast (unless `silent`) · 422 → `fieldErrors` under inputs	api.js, errors.js
Mutation	service call → toast → close modal → `reload()` or navigate	pages/dashboard/*
Search / filter	`useDebounced` 300 ms → new deps → old request aborted	useDebounced, useFetch
401	`expireSession()` → `user=null` → redirect to login	session.js, AuthContext
Logout	revoke → clear token → `user=null` → `/login`	authService, layouts
Other tab	`storage` event → `restore()`	AuthContext

PART C

Testing

Jest + Testing Library through react-scripts. The setup gotchas with React Router 7, then tests for the parts most likely to break: guards, role navigation, useFetch and error parsing.

STEP 27 Test setup with CRA 5 (Jest 27) and React Router 7

The repo deleted the default `App.test.js` and had no tests. Two small changes make React Router 7 testable under CRA's Jest 27.

src/setupTests.js

```
// jest-dom adds custom jest matchers for asserting on DOM nodes: toBeInTheDocument(), toHaveTextContent()...
import '@testing-library/jest-dom';
import { TextEncoder, TextDecoder } from 'util';

// React Router 7 needs these; CRA's jsdom environment does not provide them
Object.assign(global, { TextEncoder, TextDecoder });
```

package.json (added key)

```
{
  "jest": {
    "moduleNameMapper": {
      "^react-router/dom$": "<rootDir>/node_modules/react-router/dist/development/dom-export.js"
    }
  }
}
```

Error	Cause	Fix
`Cannot find module 'react-router/dom'`	Jest 27 ignores the `exports` map of `react-router`	`moduleNameMapper` above
`TextEncoder is not defined`	jsdom in Jest 27 lacks it, React Router 7 uses it	polyfill in `setupTests.js`
`Cannot use import statement outside a module` (axios)	only if a test imports the real `services/api.js`	mock the service module, or map `^axios$` to `axios/dist/node/axios.cjs`

terminal

```
npm test                              # watch mode
CI=true npm test -- --watchAll=false  # single run (CI)
npm test -- --coverage --watchAll=false
```

NOTE

All four suites below (7 tests) pass with `react-scripts test` on this codebase. `CI=true npm run build` also passes, which means zero ESLint warnings.

STEP 28 Unit tests: guards, navigation, useFetch, errors

Mock `useAuth` to test the guards without a backend. Use deferred promises to prove that the race fix works.

src/components/guards.test.jsx

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import { MemoryRouter, Route, Routes } from 'react-router-dom';
import { RequireAuth } from './guards';
import { useAuth } from '../context/AuthContext';

jest.mock('../context/AuthContext', () => ({ useAuth: jest.fn() }));

function renderAt(path, roles) {
  return render(
    <MemoryRouter initialEntries={[path]}>
      <Routes>
        <Route path="/login" element={<p>login page</p>} />
        <Route element={<RequireAuth roles={roles} />}>
          <Route path="/dashboard/users" element={<p>users page</p>} />
        </Route>
      </Routes>
    </MemoryRouter>
  );
}

test('guest is redirected to /login', () => {
  useAuth.mockReturnValue({ loading: false, isAuthenticated: false });
  renderAt('/dashboard/users');
  expect(screen.getByText('login page')).toBeInTheDocument();
});

test('wrong role gets a 403, right role gets the page', () => {
  useAuth.mockReturnValue({ loading: false, isAuthenticated: true, role: 'student' });
  const { unmount } = renderAt('/dashboard/users', ['admin']);
  expect(screen.getByText('403')).toBeInTheDocument();
  unmount();

  useAuth.mockReturnValue({ loading: false, isAuthenticated: true, role: 'admin' });
  renderAt('/dashboard/users', ['admin']);
  expect(screen.getByText('users page')).toBeInTheDocument();
});
```

src/hooks/useFetch.test.js

```
import { renderHook, waitFor } from '@testing-library/react';
import useFetch from './useFetch';

/** A promise we resolve by hand, to control which response arrives first. */
const deferred = () => {
  let resolve;
  const promise = new Promise((r) => { resolve = r; });
  return { promise, resolve };
};

test('a slow old response never overwrites a newer one', async () => {
  const slow = deferred();
  const fast = deferred();
  const loader = jest.fn().mockReturnValueOnce(slow.promise).mockReturnValueOnce(fast.promise);

  const { result, rerender } = renderHook(({ q }) => useFetch(loader, [q]), { initialProps: { q: 'a' } });
  rerender({ q: 'ab' }); // second request starts and aborts the first

  fast.resolve('results for ab');
  await waitFor(() => expect(result.current.data).toBe('results for ab'));
```

```
  slow.resolve('results for a'); // arrives late
  await new Promise((r) => setTimeout(r, 0));
  expect(result.current.data).toBe('results for ab');
  expect(loader.mock.calls[0][0].signal.aborted).toBe(true);
});
```

src/config/navigation.test.js

```
import { navForRole } from './navigation';

const labels = (role) => navForRole(role).flatMap((g) => g.items.map((i) => i.label));

test('admin sees administration, not "My children"', () => {
  expect(labels('admin')).toEqual(expect.arrayContaining(['Users', 'Enrollments']));
  expect(labels('admin')).not.toContain('My children');
});

test('parent sees "My children" but no admin group', () => {
  expect(labels('parent')).toContain('My children');
  expect(navForRole('parent').map((g) => g.heading)).not.toContain('Administration');
});
```

src/services/errors.test.js

```
import { fieldErrors, getErrorMessage } from './errors';

const laravel422 = { response: { status: 422, data: { message: 'Invalid.', errors: { email: ['Taken.', 'Other'] } }
} };

test('fieldErrors keeps the first message per field', () => {
  expect(fieldErrors(laravel422)).toEqual({ email: 'Taken.' });
  expect(fieldErrors(new Error('boom'))).toEqual({});
});

test('getErrorMessage prefers the first validation error, then message, then network', () => {
  expect(getErrorMessage(laravel422)).toBe('Taken.');
  expect(getErrorMessage({ response: { data: { message: 'Nope' } } })).toBe('Nope');
  expect(getErrorMessage({ request: {} })).toBe('Cannot reach the server.');
});
```

TIP

Next steps: mock `services` with `jest.mock('../../services')` to test a page (e.g. that Register shows the 422 field error), and add Playwright for an end-to-end login → enroll → logout flow against the real API.

PART D

Reference

Endpoints, env vars, commands, version notes and every fix compared with the repo.

STEP 29 API endpoints used by the frontend

This is the Laravel API surface from `backend/routes/api.php`, mapped to the service method that calls each endpoint.

Method	Path (prefix /api)	Auth	Called by
POST	/auth/register	public	`authService.register`
POST	/auth/login	public	`authService.login` (422 on bad credentials)
GET	/auth/roles	public	`authService.getRoles` (student, parent only)
GET	/auth/parents	public	`authService.getParents` (id + name)
POST	/auth/logout	token	`authService.logout`
GET	/auth/me	token	`authService.me`
PUT	/auth/profile	token	`authService.updateProfile`
GET	/courses · /courses/:id	public (token optional)	`courseService.getAll / getById`
POST · PUT · DELETE	/courses[/:id]	admin	`courseService.create / update / delete`
GET	/categories · /categories/:id	public	`categoryService.getAll / getById`
POST · PUT · DELETE	/categories[/:id]	admin	`categoryService.create / update / delete`
GET	/dashboard	token	`dashboardService.getStats` (shape per role)
GET · POST · DELETE	/users[/:id]	admin	`userService.getAll / create / delete`
GET · PUT	/users/:id	token (self or admin)	`userService.getById / update`
GET · POST · PUT · DELETE	/enrollments[/:id]	token (per-record checks)	`enrollmentService.*`

STEP 30 Environment variables & commands

The frontend needs one variable. The backend variables listed here are the ones that affect the SPA.

Variable	File	Default	Purpose
`REACT_APP_API_URL`	frontend `.env`	`http://127.0.0.1:8000/api`	API base URL, inlined at build time
`CORS_ALLOWED_ORIGINS`	backend `.env`	`http://localhost:3000,http://127.0.0.1:3000`	origins allowed to call the API
`SANCTUM_EXPIRATION`	backend `.env`	`10080` (7 days, minutes)	token lifetime → when the 401 flow kicks in
`SANCTUM_STATEFUL_DOMAINS`	backend `.env`	Laravel default	only used if you switch to SPA cookie auth

Command	What it does
`npm start`	dev server on :3000 with hot reload
`npm test` / `CI=true npm test -- --watchAll=false`	tests, watch / single run
`npm run build`	production bundle in `build/` (`CI=true` turns lint warnings into errors)
`npx serve -s build`	serve the build with SPA fallback (deep links → index.html)
`cd ../backend && php artisan serve`	API on :8000
`php artisan migrate:fresh --seed`	reset the DB with demo accounts (`admin@school.com` / `password` …)

WATCH OUT

Any static host must rewrite unknown paths to `index.html`, or refreshing on `/dashboard/courses` returns a 404. On Nginx: `try_files $uri /index.html;`. On Netlify: `/* /index.html 200`.

STEP 31 Version notes & upgrade path

These are the moving parts of this stack, and what changes when you upgrade each one.

Piece	Status / change	What to do here
Create React App	Officially deprecated (Feb 2025), no more updates; stuck on Jest 27 / webpack 5 config	Move to **Vite**: `index.html` at the project root with `<script type="module" src="/src/index.jsx">`; `REACT_APP_API_URL` → `VITE_API_URL` read through `import.meta.env.VITE_API_URL`; `.js` files that contain JSX → `.jsx`; tests → **Vitest** (no more `moduleNameMapper` hack)
React Router 7	`react-router-dom` only re-exports `react-router`	Import from `'react-router'` in new code. Library mode (used here) needs no other change. Data/framework mode adds loaders and actions
React 19	`ref` is a normal prop, `<Context>` works as a provider, `use()`	Nothing here depends on removed APIs. `forwardRef` is not needed for new components
Tailwind 3 → 4	CSS-first config	`@tailwind base/components/utilities` → `@import "tailwindcss";`; plugin `tailwindcss` → `@tailwindcss/postcss` (or `@tailwindcss/vite`); the `brand` palette moves to `@theme { --color-brand-600: … }`; automate it with `npx @tailwindcss/upgrade`

Security gap (frontend-visible)	Risk	Fix
Bearer token in `localStorage`	XSS can steal it	strict CSP, no `dangerouslySetInnerHTML`, or Sanctum SPA cookies
`GET /auth/parents` is public	anyone can list every parent's name	link a child by parent email or invite code, or require auth
Placeholder images from `picsum.photos`	third-party request on every card (IP, referrer)	ship a local placeholder; set `referrerPolicy="no-referrer"`
Image URL entered by admins	any host, mixed content	validate `https` URLs on the API, or upload to storage
Register role select	client-controlled	API must only accept `student` / `parent`

STEP 32 Bugs & gaps fixed vs. the repo

Every change in this guide compared with the current code. The fixed files were compiled (`CI=true react-scripts build`) and tested (7 Jest tests).

File	Problem	Fix	Step
`services/api.js`	401 → `window.location.assign('/login')`: full reload, `from` lost, context still holds the user	`expireSession()` event → AuthContext → guards	07
`services/api.js`	aborted requests would trigger "Cannot reach the server"	`axios.isCancel` → no toast	07
`services/index.js`	user cached in `localStorage`, never read (`getCurrentUser`, `isAuthenticated` unused)	removed; user only in React state	06
`hooks/useFetch.js`	no cancellation: an old slow response overwrites newer search results	AbortController + ignore stale results (tested)	10
`AuthContext.jsx`	no cross-tab sync; value rebuilt every render; role checks as functions	`storage` listener, `useMemo`, boolean flags + `role`	12
`Login.jsx` / `Register.jsx`	page `navigate()` races `GuestOnly`'s redirect	only `GuestOnly` redirects (to `from`)	14
`Register.jsx`	`password_confirmation` error never shown	`error` on the Confirm field	17
`CourseForm.jsx`	edit always sends `instructor_id: user.id`: instructor silently replaced	only on create; `price` sent as a number	20
`Users.jsx`	parent dropdown and admin role built from the **filtered** rows	`/auth/parents` + `me.role`	22
`Children.jsx`	`Math.round(null)` → "avg grade 0%"	"—" when there is no grade	24
`ui/index.jsx`	`Avatar` crashes on `name: null`	`(name || '?')`	16
5 pages	copied 422 → field-error mapping	`fieldErrors()` in `services/errors.js`	08
`catalog.jsx`, `Users.jsx`	two copies of the debounce effect	`hooks/useDebounced.js`	10
Tests	none (default test deleted); RR7 cannot load under CRA's Jest	setup fixes + 4 suites	27–28
`README.md`	says Tailwind v4, installed 3.4.19	fix the doc or migrate	04, 31

> **NOTE**
> Still open, because these need backend changes: an `enrollments_per_month` aggregate for the admin chart (Step 23), a capacity check inside the enrolment transaction (Step 19), and the security gaps listed in Step 31.